

Storage Alternatives for Large Structured State Spaces^{*}

Gianfranco Ciardo and Andrew S. Miner

College of William and Mary, Williamsburg VA 23187-8795, USA

Abstract

We consider the problem of storing and searching a large state space obtained from a high-level model such as a queueing network or a Petri net. After reviewing the traditional technique based on a single search tree, we demonstrate how an approach based on multiple levels of search trees offers advantages in both memory and execution complexity. Further execution time improvements are obtained by exploiting the concept of “event locality”. We apply our technique to three large parametric models, and give detailed experimental results.

1 Introduction

Extremely complex systems are increasingly common. Various types of high-level models are used to describe them, study them, and forecast the effect of possible modifications. Unfortunately, the logical and dynamic analysis of these models is often hampered by the combinatorial explosion of their state spaces, a problem inherent with discrete-state systems.

It is quite common to use exact methods in the early stages of a system’s design, when studying rough models over a wide range of parameter combinations, then focus on more detailed models using approximate methods or simulation. Indeed, many approximate decomposition approaches use exact methods at the submodel level, thus introducing errors only when exchanging or combining (exact) results from different submodels. Then, the ability of applying exact methods to larger models will normally result in better approximations: it is likely that the study of a large model decomposed into four medium-size submodels, each solved with exact methods, will be more accurate than that of the same model decomposed into ten small-size submodels.

The first problem when applying exact methods is the generation and storage of the state space. After that, if the goal of the study is performance or reliability, the model is then used to generate a stochastic process, often a continuous-time Markov chain (CTMC), which is solved numerically for steady-state or transient measures. The state space can be of interest in itself, though, since it can be used to answer questions such as existence of deadlocks and livelocks or liveness.

^{*} This research was partially supported by the National Aeronautics and Space Administration under NASA Contract No. NAS1-19480 and by the Center for Advanced Computing and Communication under Contract 96-SC-NSF-1011

In this paper, we focus on techniques to store and search the state space. However, while we do not discuss the solution of the stochastic process explicitly, our results apply not only to the “logical” analysis of the model, but even more so to its “stochastic analysis” (indeed, this was our initial motivation). This is particularly true given the recent developments on the solution of complex Markov models using Kronecker operators [2] which do not require to store the infinitesimal generator of the CTMC explicitly. In this case, the storage bottleneck is indeed due to the state space and the probability vectors allocated when computing the numerical solution.

Sections 2 and 3 define the type of high-level formalisms we consider and discuss the reachability set and its storage, respectively. Our main contributions are in Sections 4 and 5, where we introduce a multilevel data structure and show how it can be used to save both memory and execution time when building the reachability set. Finally, Section 6 contains our final remarks.

2 High-Level Model Description

We implemented our techniques in the generalized stochastic Petri net (GSPN) formalism. However, these techniques can be applied to any model expressed in a “high-level formalism” that defines:

- The “potential set”, $\hat{\mathcal{R}}$. This is a discrete set, assumed finite, to which the states of the model must belong².
- A finite set of possible events, $\mathcal{E}$.
- An initial state, $\mathbf{i}^0 \in \hat{\mathcal{R}}$.
- A boolean function defining whether an event is active (can occur) in a state, $Active: \mathcal{E} \times \hat{\mathcal{R}} \rightarrow \{True, False\}$.
- A function defining the effect of the occurrence of an active event on a state: $New: \mathcal{E} \times \hat{\mathcal{R}} \rightarrow \hat{\mathcal{R}}$.

Fig. 1 and 2 show the two GSPNs used throughout this paper to illustrate the effect of our techniques (ignore for the moment the dashed boxes). The first GSPN models a kanban system, from [2]. It is composed of four instances of essentially the same sub-GSPN. The synchronizing transitions t_{synch_1} and t_{synch_2} can be either both timed or both immediate, we indicate the two resulting models as kanban-timed and kanban-immediate. The second GSPN models a flexible manufacturing system, from [3], except that the cardinality of all arcs is constant, unlike the original model (this does not affect the number of reachable markings). We indicate this model as FMS. We do not describe these models in more detail, the interested reader is referred to the original publications where they appeared. In all cases, their initial marking is a function of a parameter N .

² A point about notation: we denote sets by upper case calligraphic letters. Lower and upper case bold letters denote vector and matrices, respectively. $\eta(\mathbf{A})$ is the number of nonzero entries in a matrix $\mathbf{A}$; $\mathbf{A}_{i,j}$ is the entry in row i and column j of $\mathbf{A}$; $\mathbf{A}_{\mathcal{X},\mathcal{Y}}$ is the submatrix of $\mathbf{A}$ corresponding to the set of rows $\mathcal{X}$ and the set of columns $\mathcal{Y}$.

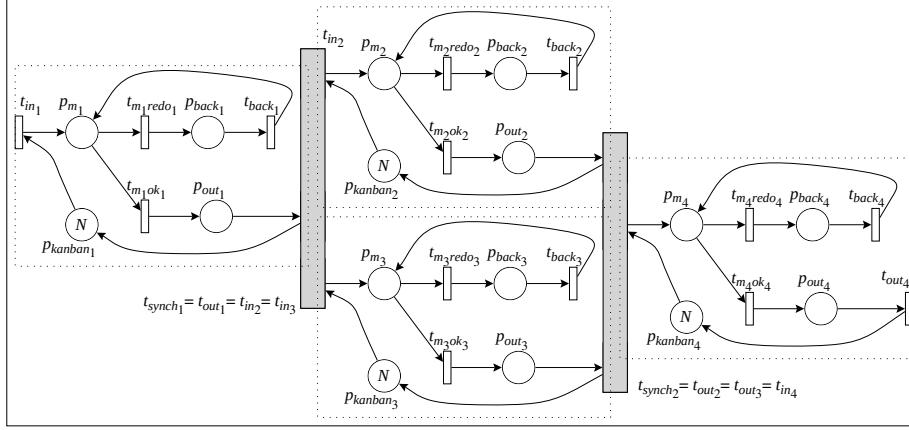


Fig. 1. The GSPN of a Kanban system.

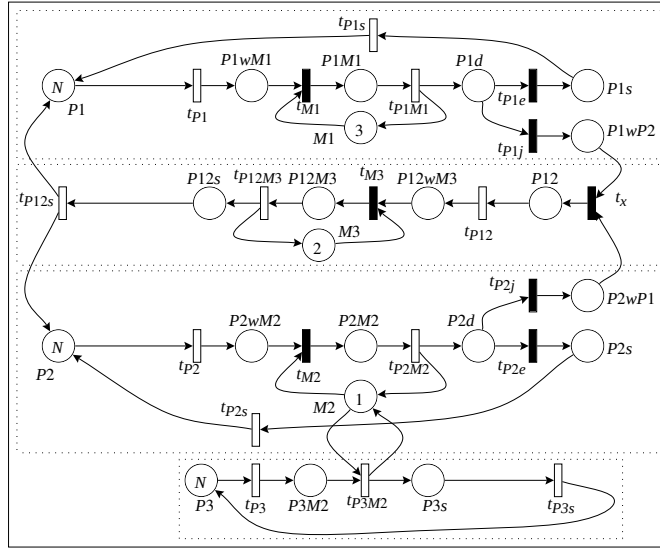


Fig. 2. The GSPN of a FMS system.

3 The Reachability Set

From the high-level description, we can build the “reachability set”, $\mathcal{R} \subseteq \hat{\mathcal{R}}$. This is the set of states that can be reached from $\mathbf{i}^0$ through the occurrence of any sequence of enabled events. Formally, $\mathcal{R}$ is the smallest subset of $\hat{\mathcal{R}}$ satisfying:

1. $\mathbf{i}^0 \in \mathcal{R}$, and
2. $\mathbf{i} \in \mathcal{R} \wedge \exists e \in \mathcal{E}, \text{Active}(e, \mathbf{i}) = \text{True} \wedge \mathbf{i}' = \text{New}(e, \mathbf{i}) \Rightarrow \mathbf{i}' \in \mathcal{R}$.

```

BuildRS(in: Events, Active, New; out:  $\mathcal{R}$ );
1.  $\mathcal{R} \leftarrow \emptyset$ ; /*  $\mathcal{R}$ : states explored so far */
2.  $\mathcal{U} \leftarrow \{s^0\}$ ; /*  $\mathcal{U}$ : states found but not yet explored */
3. while  $\mathcal{U} \neq \emptyset$  do
4.    $\mathbf{i} \leftarrow \text{ChooseRemove}(\mathcal{U})$ ;
5.    $\mathcal{R} \leftarrow \mathcal{R} \cup \{\mathbf{i}\}$ ;
6.   for each  $e \in \mathcal{E}$  s.t.  $\text{Active}(e, \mathbf{i}) = \text{True}$  do
7.      $\mathbf{j} \leftarrow \text{New}(e, \mathbf{i})$ ;
8.      $\text{SearchInsert}(\mathbf{j}, \mathcal{R} \cup \mathcal{U}, \mathcal{U})$ ;

```

Fig. 3. Procedure *BuildRS*

While a high-level model usually specifies other information as well (the stochastic timing of the events, the measures that should be computed when solving the model, etc.), the above description is sufficient for now, and it is not tied to any particular formalism.

$\mathcal{R}$ can be generated using the state-space exploration procedure *BuildRS* shown in Fig. 3, which terminates if $\mathcal{R}$ is finite. This is a search of the graph implicitly defined by the model. Function *ChooseRemove* chooses an element from its argument (a set of states), removes it, and returns it. Procedure *SearchInsert* searches the first argument (a state) in the second argument (a set of states) and, if not found, it inserts the state in its third argument (also a set of states).

The “reachability graph” $(\mathcal{R}, \mathcal{A})$ has as nodes the reachable states, and an arc from $\mathbf{i}$ to $\mathbf{j}$ iff there is an event e such that $\text{Active}(e, \mathbf{i}) = \text{True}$ and $\text{New}(e, \mathbf{i}) = \mathbf{j}$. Depending on the type of analysis, the arc might be labeled with e ; this might result in a multigraph, if multiple events can cause the same change of state. While we focus on $\mathcal{R}$ alone, the size of $\mathcal{A}$ affects the complexity of *BuildRS*.

A total order can be defined over the elements of $\hat{\mathcal{R}}$: $\mathbf{i} < \mathbf{j}$ iff $\mathbf{i}$ precedes $\mathbf{j}$ in lexical order. We can then define a function *Compare* : $(\hat{\mathcal{R}} \times \hat{\mathcal{R}}) \rightarrow \{-1, 0, 1\}$ returning -1 if the first argument precedes the second one, 1 if it follows it, and 0 if the two arguments are identical. This order allows the efficient search and insertion of a state during *BuildRS*. Once $\mathcal{R}$ has been built, we can define a bijection $\Psi : \mathcal{R} \rightarrow \{0, \dots, |\mathcal{R}| - 1\}$, such that $\Psi(\mathbf{i}) < \Psi(\mathbf{j})$ iff $\mathbf{i} < \mathbf{j}$.

Common techniques to store and search the sets $\mathcal{R}$ and $\mathcal{U}$ include hashing and search trees. Hashing would work reasonably well if we had a good bound on the final size of the reachability set $\mathcal{R}$, but this is not usually the case. We prefer search trees: when kept balanced, they have more predictable behavior than hashing.

We consider two alternatives, both based on binary search trees: splay trees [4] and AVL trees [5]. The execution complexity³ of *BuildRS* when using either

³ We leave known constants inside the big-Oh notation to stress that they do make a difference in practice, even if they are formally redundant.

method is

$$O(|\mathcal{R}| \cdot |\mathcal{E}| \cdot C_{Active} + |\mathcal{A}| \cdot (C_{New} + \log |\mathcal{R}| \cdot C_{Compare})),$$

where C_x is the (average) cost to a call to procedure x . For AVL trees, the storage complexity, expressed in bits, is

$$O(|\mathcal{R}| \cdot (B_{state} + 2B_{pointer} + 2)),$$

where B_{state} is the (average) number of bits to store a state, $B_{pointer}$ is the number of bits for a pointer. The additional two bits are used to store the node balance information. For splay trees, the complexity is the same, except for the two-bit balance, which is not required.

Regarding the use of binary search trees in *BuildRS*, we have another choice to make. We can use a single tree to store $\mathcal{R} \cup \mathcal{U}$ in *BuildRS* as shown in Fig. 4(a,b,c), or two separate trees, as in Fig. 4(d). Using a single tree has some advantages:

- No deletion, possibly requiring a rebalancing, is needed when moving a state from $\mathcal{U}$ to $\mathcal{R}$.
- Procedure *SearchInsert* performs a single search requiring $O(\log |\mathcal{R} \cup \mathcal{U}|)$ comparisons, instead of two searches in the two trees for $\mathcal{R}$ and $\mathcal{U}$, which overall require more comparisons (up to twice as many), since

$$\log |\mathcal{R}| + \log |\mathcal{U}| = \log(|\mathcal{R}| \cdot |\mathcal{U}|) > \log(|\mathcal{R}| + |\mathcal{U}|) = \log(|\mathcal{R} \cup \mathcal{U}|)$$

(of course, we mean “the current $\mathcal{R}$ and $\mathcal{U}$ ” in the above expressions).

However, it also has disadvantages. As explored and unexplored states are stored together in a single tree, there must be a way to identify and access the nodes of $\mathcal{U}$. We can accomplish this by:

1. Maintaining a separate linked list pointing to the unexplored states. This requires an additional $O(2|\mathcal{U}| \cdot B_{pointer})$ bits, as shown in Fig. 4(a).
2. Storing an additional pointer in each tree node, pointing to the next unexplored state. This requires an additional $O(|\mathcal{R}| \cdot B_{pointer})$ bits, as shown in Fig. 4(b).
3. Storing the markings in a dynamic array structure. In this case, the nodes of the tree contain indices to the array of markings, instead of the markings themselves. If markings are added to the array in the order they are discovered, $\mathcal{R}$ occupies the beginning of the array, while $\mathcal{U}$ occupies the end. This requires an additional $O(|\mathcal{R}| \cdot B_{index})$ bits, where B_{index} is the number of bits required to index the array of markings, as shown in Fig. 4(c)

The size of $\mathcal{R}$ for our three models is shown in Table 1, as a function of N . Since the models are GSPNs, they give rise to “vanishing markings”, that is, markings enabling only immediate transitions, in which the sojourn time is zero. In all our experiments, these markings are eliminated “on the fly” so that all our figures regarding $\mathcal{R}$ reflect only the “tangible” reachable markings.

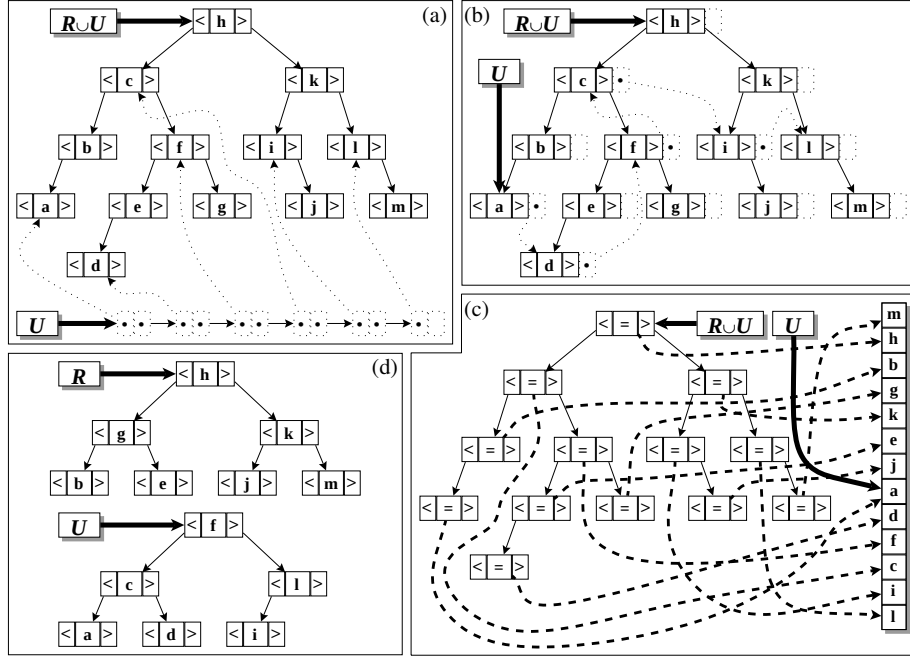


Fig. 4. Four simple storage schemes for $\mathcal{R}$ and $\mathcal{U}$.

N	$ \mathcal{R} $ for kanban-timed	$ \mathcal{R} $ for kanban-immediate	$ \mathcal{R} $ for FMS
1	160	152	54
2	4,600	3,816	810
3	58,400	41,000	6,520
4	454,475	268,475	35,910
5	2,546,432	1,270,962	152,712
6	11,261,376	4,785,536	537,768
7	—	—	1,639,440

Table 1. Reachability set size for the three models, as a function of N .

4 A Multilevel Search Tree to Store $\mathcal{R}$

We now assume that $\hat{\mathcal{R}}$ can be expressed as the Cartesian product of K “local” state spaces: $\hat{\mathcal{R}} = \mathcal{R}^0 \times \dots \times \mathcal{R}^{K-1}$, and that $\mathcal{R}^k$ is simply $\{0, \dots, n_k - 1\}$, although n_k might not be known in advance. Hence, a (global) state is a vector $\mathbf{i} \in \{0, \dots, n_0 - 1\} \times \dots \times \{0, \dots, n_{K-1} - 1\}$. For example, if the high-level model is a single-class queuing network, $\mathbf{i}_k$ could be the number of customers in queue k . However, a realistic model can easily have dozens of queues; so it might be more efficient to partition the queues into K sets, and let $\mathbf{i}_k$ be the number of

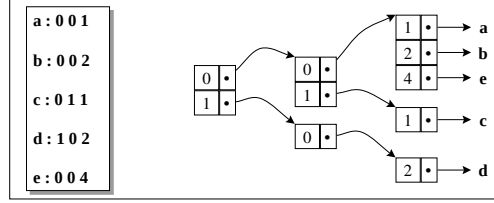


Fig. 5. Chiola's storage scheme to store the reachability set of a SPN.

possible combinations of customers into the queues of partition k . An analogous discussion applies to stochastic Petri nets (SPNs), and even to more complex models, as long as they have a finite state space.

Our lexical order can be applied just as well to $\hat{\mathcal{R}}$, hence we can define a bijection $\hat{\Psi} : \hat{\mathcal{R}} \rightarrow \{0, \dots, |\hat{\mathcal{R}}| - 1\}$, analogous to Ψ . Indeed, $\hat{\Psi}$ has a fundamental advantage over Ψ : its value equals its argument interpreted as a mixed base integer,

$$\hat{\Psi}(\mathbf{i}) = (\dots((\mathbf{i}_0)n_1 + \mathbf{i}_1)n_2 \dots)n_{K-1} + \mathbf{i}_{K-1} = \sum_{k=0}^{K-1} \mathbf{i}_k \cdot n_{k+1}^{K-1},$$

where $n_i^j = \prod_{k=i}^j n_k$. The reachability set $\mathcal{R}$ may coincide with $\hat{\mathcal{R}}$, but it is more likely to be much smaller, since many combinations of local states might not occur.

We extend work by Chiola, who defined a multilevel technique to store the reachable markings of a SPN [1]. Since the state, or marking, of a SPN with K places can be represented as a fixed-size vector $\mathbf{i} = (\mathbf{i}_0, \mathbf{i}_1, \dots, \mathbf{i}_{K-1})$, he proposed the strategy illustrated in Fig. 5, copied from [1]. In the figure, it is assumed that the SPN has three places, and that there are five reachable markings, **a** through **e**. A three-level tree is then used. The first level discriminates markings on their first component (either 0 or 1); the second level discriminates markings on their second component (also either 0 or 1) within a given first component; finally, the third level fully determines the marking using the third component (1, 2, or 4).

We instead use a multilevel approach with K levels, one for each submodel (a sub-GSPN), as shown in Fig. 6. Before explaining this data structure, we need to define two classes of sets.

- The set of reachable substates up to k :

$$\begin{aligned} \mathcal{R}_0^k &= \{(\mathbf{i}_0, \dots, \mathbf{i}_k) \in \mathcal{R}^0 \times \dots \times \mathcal{R}^k : \\ &\quad \exists(\mathbf{i}_{k+1}, \dots, \mathbf{i}_{K-1}) \in \mathcal{R}^{k+1} \times \dots \times \mathcal{R}^{K-1}, (\mathbf{i}_0, \dots, \mathbf{i}_{K-1}) \in \mathcal{R}\}. \end{aligned}$$

Clearly, $\mathcal{R}_0^{K-1}$ coincides with $\mathcal{R}$ itself, while $\mathcal{R}_0^0$ coincides with $\mathcal{R}^0$ iff every element of the local state space 0 can actually occur in some global state, and so on.

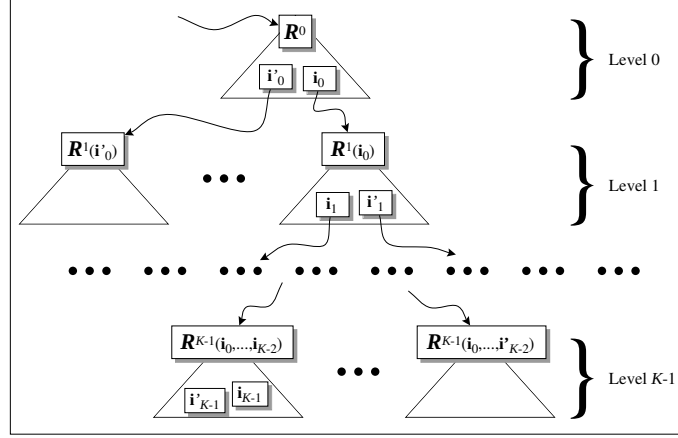


Fig. 6. A multilevel storage scheme for $\mathcal{R}$.

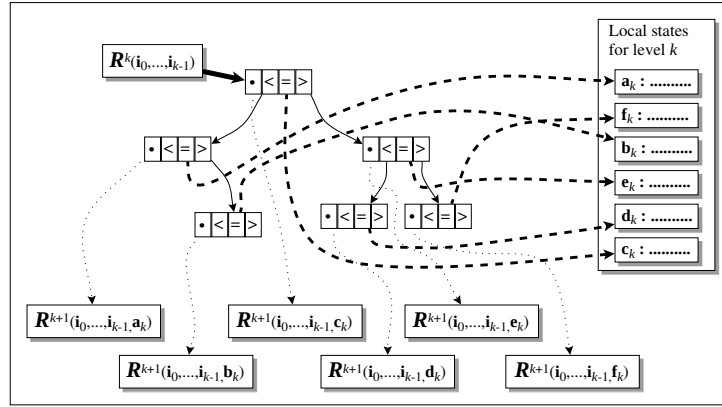


Fig. 7. A tree at level $k < K - 1$, pointing to trees at level $k + 1$.

- The set of reachable local states in $\mathcal{R}^k$ conditioned on a reachable substate $(\mathbf{i}_0, \dots, \mathbf{i}_{k-1}) \in \mathcal{R}_0^{k-1}$:

$$\mathcal{R}^k(\mathbf{i}_0, \dots, \mathbf{i}_{k-1}) = \{\mathbf{i}_k \in \mathcal{R}^k : (\mathbf{i}_0, \dots, \mathbf{i}_k) \in \mathcal{R}_0^k\}.$$

Clearly, this set, when defined (i.e., when indeed $(\mathbf{i}_0, \dots, \mathbf{i}_{k-1}) \in \mathcal{R}_0^{k-1}$), is never empty.

In Fig. 6, level $k \in \{0, \dots, K - 1\}$ contains $|\mathcal{R}_0^{k-1}|$ search trees, each one storing $\mathcal{R}^k(\mathbf{i}_0, \dots, \mathbf{i}_{k-1})$, for a different reachable substate $(\mathbf{i}_0, \dots, \mathbf{i}_{k-1}) \in \mathcal{R}_0^{k-1}$. Thus, a generic tree at level $k < K - 1$ has the structure shown in Fig. 7. In each one of its nodes, the “•” pointer points to a tree at level $k + 1$. In the drawing,

the local states for level k are stored in an unsorted dynamically extensible array, in the order in which they are found, and pointed by the “=” pointer in each node. Alternatively, it is possible to store a local state directly in the node. If the submodel is quite complex, this second alternative might be not as memory effective, since a local state can require several bytes for its encoding, which are then repeated in each node, instead of just a pointer. Furthermore, if we can assume an upper bound on the number of local states for a given submodel (e.g., 2^{16}), we can use an index (16 bits) into the local state array, instead of a pointer (usually 32 bits).

The total number of nodes used for trees at level k equals the number of reachable substates up to that level, $|\mathcal{R}_0^k|$, hence the total number of tree nodes for our multilevel data structure is

$$\sum_{k=0}^{K-1} |\mathcal{R}_0^k| > |\mathcal{R}_0^{K-1}| = |\mathcal{R}|$$

where the last expression is the number of nodes required in the standard single-level approach of Fig. 4. Two observations are in order.

First, the total number of nodes at levels 0 through $K - 2$ is a small fraction of the nodes at level $K - 1$, that is, of $|\mathcal{R}|$, provided the trees at the last level, $K - 1$, are not “too small”. This is ensured by an appropriate partitioning of a large model into submodels. In other words, only the memory used by the nodes of the trees at the last level really matters.

Second, the nodes of the trees at the last level do not require a pointer to a tree at a lower level, hence each node requires only three pointers (96 bits). Again, if we can assume a bound on the number of local states for the last submodel, $|\mathcal{R}^{K-1}| \leq 2^b$, we can in principle implement dynamic data structures that require only $3b$ bits per node, plus some overhead to implement dynamic arrays. Given a model, there is usually some freedom in choosing the number of submodels, that is, the number of levels K . We might then be able to define the last submodel in such a way that it has at most 2^8 local states, resulting in only slightly more than $3|\mathcal{R}|$ bytes to store the entire reachability set. When this is not the case, the next easily implementable step, 2^{16} , is normally more than enough. Even in this case, the total memory requirements are still much better than with a single-level implementation.

We stress that, while this discussion about using 8 or 16 bit indices seems to be excessively implementation-dependent, it is not. Only through the multilevel approach it is possible to exploit the small size of each local space. In the single-level approach, instead, the total number of states that can be indexed (pointed by a pointer) is $|\mathcal{R}|$, a number which can be easily be 10^7 , and possibly more. In this case, we are forced to use 32-bit pointers, and we can even foresee a point in the not-so-distant future where even these pointers will be insufficient, while the multilevel approach can be implemented using 8 or 16 bit indices at the last level regardless of the size of the reachability set, if $\mathcal{R}^{K-1}$ is not too large.

Just as with a single search tree, we can implement our multilevel scheme using splay or AVL trees. However, the choice between storing $\mathcal{R}$ and $\mathcal{U}$ sep-

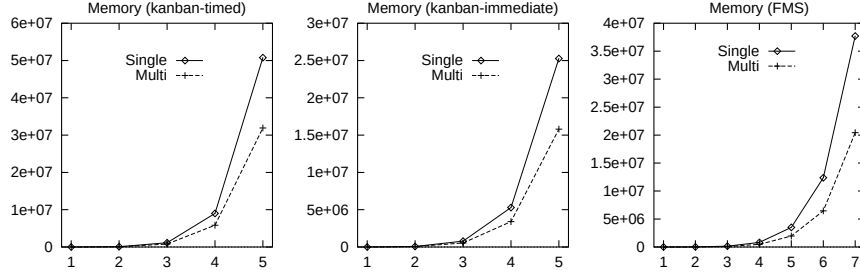


Fig. 8. Memory usage (bytes) for single vs. multilevel approach, as a function of N .

arately or as a single set has subtler implications. If we choose to store them as a single set, distinguishing between explored and unexplored states or, more precisely, being able to access the unexplored states only, still requires one of the approaches discussed for the single-level case (Fig. 4). Since only local states are stored for each level, without repetition, we can no longer use the state array approach of Fig. 4(c). The other two methods, which involve maintaining a list of unexplored states, would only allow us to access nodes in trees at level $K - 1$, which are meaningless if we do not know the entire path from level 0. In other words, we would only know the last component of each unexplored state, but not the first $K - 1$ components. To obtain all the components, we must add a backward pointer from each node (conceptually, from each tree, since all the nodes in a given tree have the same previous components) to a node in the previous level, and so on. This additional memory overhead could be substantial.

With the single-level approach, then, we store $\mathcal{R} \cup \mathcal{U}$ in a single tree, using the state array described in Fig. 4(c). With the multilevel approach, instead, we store $\mathcal{R}$ and $\mathcal{U}$ separately. In this second case, states are truly deleted from $\mathcal{U}$. However, we are free to remove states in any order we choose. With AVL trees, we can always use the balance information to choose, at each level, a state in a leaf of the tree in such a way that no rebalancing is ever required.

Fig. 8 shows the number of bytes used to store $\mathcal{R}$ and $\mathcal{U}$ for our three models, using the single vs. the multilevel approach. The dashed boxes in Fig. 1 and 2 indicate the decomposition into submodels, in all cases $K = 4$.

The multilevel approach is clearly preferable, especially for large models. Indeed, we could not generate the state space using the single-level approach for $N = 6$ for the kanban-timed model, while we could using the multilevel approach (all our experiments were run on a Sun-clone with a 55 MhZ HyperSparc processor and 128 Mbyte of RAM, without making use of virtual memory).

5 Execution Time

Several factors need to be considered when studying the impact of our approach on the execution time. First, we explored possible differences due to using splay

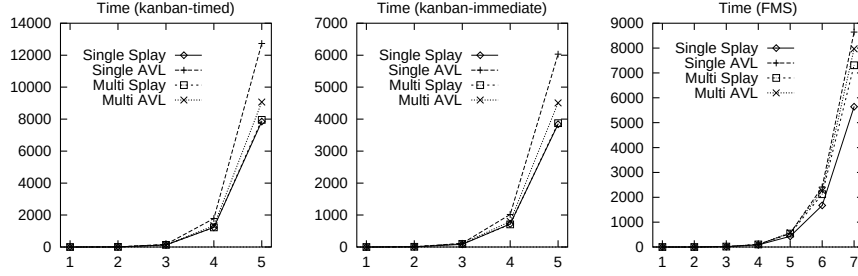


Fig. 9. Execution times (seconds) for splay vs. AVL trees, as a function of N .

trees vs. AVL trees. Fig. 9 shows that splay trees typically perform better than AVL trees. This is most likely due to the property of splay trees that recently inserted items are close to the root of the tree. In the case of multilevel trees, the difference between splay and AVL is not as great, since the trees at each level are relatively small. The effect of the peculiar splay tree behavior is further addressed in Section 5.1.

Next, we compare the differences due to using single-level vs. multilevel trees. The multilevel approach has many advantages.

Consider first the case when *BuildRS* searches for a state $\mathbf{i}$ not yet in the current $\mathcal{R}$ (the same discussion holds for $\mathcal{U}$). With a single-level tree, the search will stop only after reaching a leaf, hence $O(\log |\mathcal{R}|)$ comparisons are always performed. In the multilevel approach, instead, the search stops as soon as the substate $(\mathbf{i}_1, \dots, \mathbf{i}_k)$ is not reachable, for some $k \leq K - 1$. If all the trees at a given level are of similar size, this will require at most $O(\log |\mathcal{R}_0^k|)$ comparisons. In practice, the situation is even more favorable, because, for any level $l < k$, the tree searched, which stores $\mathcal{R}^l(\mathbf{i}_1, \dots, \mathbf{i}_{l-1})$, contains the substate $\mathbf{i}_l$ we are looking for, so the jump from level l to level $l + 1$ might occur before reaching a leaf.

If $\mathbf{i}$ is already in $\mathcal{R}$, instead, the search on the single-level tree will sometimes find $\mathbf{i}$ before reaching a leaf, but this is true also at each level of the multilevel approach, just as, in the previous case, for the levels $l < k$.

Perhaps even more important, though, regardless of whether $\mathbf{i}$ is already in the tree or not, is the complexity of each comparison. With the multilevel approach, only substates are compared, and these are normally small data structures. For GSPNs, this could be an array of 8 or 16 bit integers, one for each place in the sub-GSPN (memory usage is not an issue, since each local state for level k is stored only once). On the other hand, with the single-level approach, entire states are compared. If the states are stored as arrays, the comparison can stop as soon as one component differs in the two arrays. However, as the search progresses down the tree, we compare states that are increasingly close in lexical order, hence likely to have the same first few components; this implies that comparisons further away from the root tend to be more expensive.

Furthermore, storing states as full arrays is seldom advisable. For example, in SMART, a sophisticated packing is used, on a state-by-state base, to reduce the memory requirements. In this case, the state in the node needs to be “unpacked” before comparing it with $\mathbf{i}$, thus effectively examining every component of the state.

As Fig. 9 indicates, a multilevel AVL tree typically performs better than a single-level AVL tree; while a single-level splay tree performs just as well or better than a multilevel splay tree. The ability of a single-level splay tree to find recently inserted states quickly and the inefficiency of splay deletion, which is required for the multilevel approach, are possible explanations for this difference.

5.1 Exploiting event locality

Our multilevel search tree has an additional advantage which can be exploited to further speed up execution. To illustrate the idea, we define two functions:

- *LocalSet*(k, I_{k-1}), which, given a submodel index $k, 0 \leq k < K$, and a pointer I_{k-1} to a reachable substate $(\mathbf{i}_0, \dots, \mathbf{i}_{k-1}) \in \mathcal{R}_0^{k-1}$, returns a pointer to the tree containing the set $\mathcal{R}^k(\mathbf{i}_0, \dots, \mathbf{i}_{k-1})$.
- *LocalIndex*($k, I_{k-1}, \mathbf{i}_k$), which, given a submodel index $k, 0 \leq k < K$, a pointer I_{k-1} to a reachable substate $(\mathbf{i}_0, \dots, \mathbf{i}_{k-1}) \in \mathcal{R}_0^{k-1}$, and a local state index $\mathbf{i}_k \in \mathcal{R}^k$, returns the pointer I_k to substate $(\mathbf{i}_0, \dots, \mathbf{i}_k)$, if reachable, null otherwise.

Given our data structure, “a pointer I_k to a reachable substate $(\mathbf{i}_0, \dots, \mathbf{i}_k) \in \mathcal{R}_0^k$ ” points to the node corresponding to the local state $\mathbf{i}_k$ in the tree containing the set $\mathcal{R}^k(\mathbf{i}_0, \dots, \mathbf{i}_{k-1})$.

To find whether state $\mathbf{i}$ is in (the current) $\mathcal{R}$, we can then use a sequence of K function calls

$$LocalIndex(K-1, \underbrace{LocalIndex(K-2, \dots, LocalIndex(0, \text{null}, \mathbf{i}_0), \dots, \mathbf{i}_{K-2})}_{I_{K-2}}, \mathbf{i}_{K-1}).$$

The overall cost of these K calls is exactly what we just discussed when comparing the single and multilevel approaches. However, if $\mathbf{i}$ is reachable and we now wanted to find the index of a state $\mathbf{i}'$ differing from $\mathbf{i}$ only in its last position, we could do so with a single call $LocalIndex(K-1, I_{K-2}, \mathbf{i}'_{K-1})$, provided we saved the value I_{K-2} , the index of the reachable substate $(\mathbf{i}_0, \dots, \mathbf{i}_{K-2})$. A similar argument applies to other values of k . Only for $k=0$ do we need to perform an entirely new search.

We can then exploit this “locality” by considering the possible effects of the events on the state. Given $\hat{\mathcal{R}} = \mathcal{R}^0 \times \dots \times \mathcal{R}^{K-1}$, we can partition $\mathcal{E}$ into $\mathcal{E}^0, \dots, \mathcal{E}^{K-1}$, such that

$$e \in \mathcal{E}^k \Leftrightarrow \left(\forall \mathbf{i} \in \hat{\mathcal{R}}, Active(e, \mathbf{i}) = True \wedge \mathbf{j} = New(e, \mathbf{i}) \Rightarrow \forall l, 0 \leq l < k, \mathbf{i}_l = \mathbf{j}_l \right),$$

that is, events in $\mathcal{E}^k$ can only change local states k or higher. For example, for GSPNs, this is achieved by assigning to $\mathcal{E}^k$ all the transitions whose enabling and

firing effect on the state depends only on places in sub-GSPN k , and possibly in sub-GSPNs $k + 1$ through $K - 1$, but not in sub-GSPNs 1 through $k - 1$. This can be easily accomplished through an automatic inspection even in the presence of guards and inhibitor and variable cardinality arcs (although in this case the partition might be overly conservative, that is, it might assign a transition to a level k when it could have been assigned to a level greater than k).

When exploring state $\mathbf{i}$ in procedure *BuildRS*, we remove $\mathbf{i}$ from $\mathcal{U}$, we insert it into $\mathcal{R}$, and we record the sequence of pointers $I_{-1} = \text{null}, I_0, \dots, I_{K-2}$, to the reachable substates $(\mathbf{i}_0), \dots, (\mathbf{i}_0, \dots, \mathbf{i}_{K-2})$. Then, we can examine the events in $\mathcal{E}$ in any order. As soon as we find an enabled event $e \in \mathcal{E}^k$, we generate $\mathbf{i}' = \text{New}(e, \mathbf{i})$, and we are guaranteed that its components 0 through $k - 1$ are identical to those of $\mathbf{i}$. Hence, we can search for it starting at I_{k-1} , that is, we only need to perform the $K - k$ calls

$$\text{LocalIndex}(K - 1, \dots, \text{LocalIndex}(k + 1, \text{LocalIndex}(k, I_{k-1}, \mathbf{i}_k), \mathbf{i}_{k+1}), \dots, \mathbf{i}_{K-1}).$$

The savings due to these accelerated searches depend on the particular model and the partition chosen for it. We show the results for our three models in Fig. 10. In the case of AVL trees, execution time savings of 20% are achieved, while the reduction is more modest, from 5 to 10%, for splay trees. Only in the FMS model does the single level approach (with splay trees) perform better than the multilevel approach with the exploitation of locality.

The comparatively good performance of the single level approach with splay trees can be attributed to our implementation choice of storing $\mathcal{R}$ and $\mathcal{U}$ separately in the multilevel approach, resulting in the lowest storage requirements, but higher overhead. We also observe that it is exactly the inherent properties of splay trees that make them perform well: after searching for a state $\mathbf{i}$, searches for states lexicographically close to it will result in few comparisons. This achieves an effect similar to that obtained by locality, but only if the order of the events explored from state $\mathbf{i}$ is “lucky”: if $\mathbf{i}$ reaches both $\mathbf{i}'$ and $\mathbf{i}''$, and $\mathbf{i} > \mathbf{i}' > \mathbf{i}''$, it is best if $\mathbf{i}'$ is searched before $\mathbf{i}''$. We verified this intuition by generating random orderings in which the transitions in our three models are considered for exploration. In the case of the single level approach with splay trees, runtime variations were as high as 12%, while much smaller effects were observed in the other cases.

6 Conclusion

We have presented a detailed analysis of a multilevel data structure for storing and searching the large set of states reachable in some high-level model. Memory usage, which is normally the main concern, is greatly reduced. In some cases, this reduction is achieved not at the expense of, but in conjunction with, execution efficiency. A further technique based on event locality achieves an additional reduction in the execution time, at absolutely no cost in terms of memory. The results substantially increase the size of the reachability sets that can be managed.

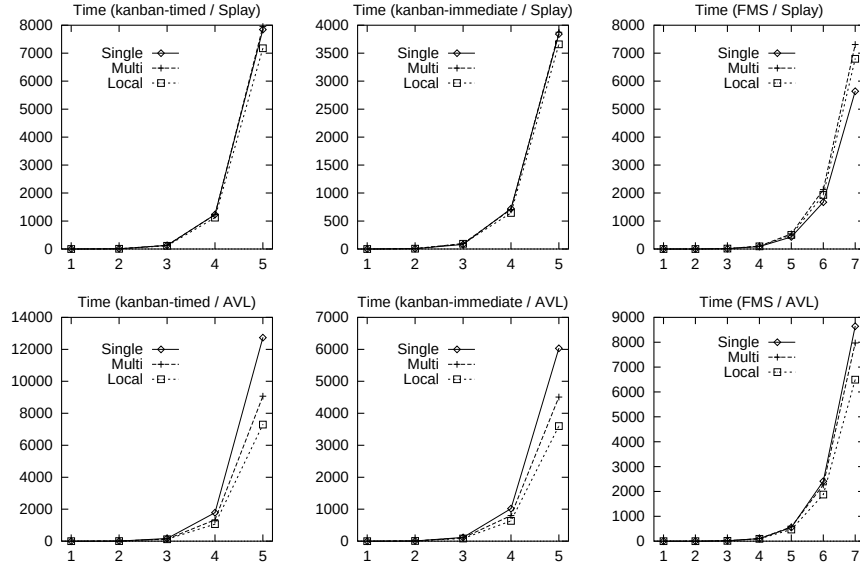


Fig. 10. Execution times (seconds) using splay or AVL trees, as a function of N .

In the future, we will investigate how to use our approach in a distributed fashion, where N cooperating processes explore different portions of the reachability set. In particular, we plan to apply the data structure we presented and to explore ways to balance the load among the cooperating processes automatically.

References

1. G. Chiola. Compiling techniques for the analysis of stochastic Petri nets. In *Proc. 4th Int. Conf. on Modelling Techniques and Tools for Performance Evaluation*, pages 13–27, 1989.
2. G. Ciardo and M. Tilgner. On the use of Kronecker operators for the solution of generalized stochastic Petri nets. ICASE Report 96-35, Institute for Computer Applications in Science and Engineering, Hampton, VA, 1996.
3. G. Ciardo and K. S. Trivedi. A decomposition approach for stochastic reward net models. *Perf. Eval.*, 18(1):37–59, 1993.
4. T. H. Cormen, C. E. Leiserson, and R. L. Rivest. *Introduction to algorithms*. The MIT Press, Cambridge, MA, 1990.
5. D. E. Knuth. *Sorting and Searching*. Addison-Wesley, Reading, MA, 1973.